

Project Assignment: AI-Powered Company Knowledge Assistant

Overview

Build an AI-powered Company Knowledge Assistant that helps employees find information from company documents such as policies, training guides, onboarding documents, and SOPs.

The application should allow users to upload documents and ask questions in natural language. The system should retrieve relevant information from the uploaded documents and generate accurate answers using a Large Language Model (LLM).

Objective

Develop a document-based AI assistant that:

- Accepts company documents as input
 - Extracts and processes document content
 - Searches relevant information efficiently
 - Generates context-aware responses
 - Displays the source document used for answering
-

Technology Stack

Mandatory

- Python
- Pandas
- Streamlit
- Google Gemini API or OpenAI API
- Sentence Transformers

Recommended Libraries

- PyPDF2
- python-docx
- pandas
- sentence-transformers
- streamlit
- python-dotenv

Functional Requirements

1. Document Upload Module

The application should allow users to upload:

- PDF files
- DOCX files
- TXT files

Uploaded files should be stored locally inside a dedicated folder.

Example:

documents/

- EmployeeHandbook.pdf
- LeavePolicy.pdf
- TrainingGuide.docx

2. Document Processing Module

Extract text from all uploaded documents.

Store extracted data using Pandas DataFrames.

Example Data Structure:

File Name	Content
LeavePolicy.pdf	Extracted Text
EmployeeHandbook.pdf	Extracted Text

3. Text Chunking

Large documents should be divided into smaller chunks for efficient searching and retrieval.

The chunking strategy should be documented in the README.

4. Semantic Search Engine

Generate embeddings for document chunks using Sentence Transformers.

When a user asks a question:

Example:

"How many casual leaves are allowed?"

The system should identify and retrieve the most relevant content from the uploaded documents.

5. AI Answer Generation

Integrate an LLM such as Gemini or OpenAI.

The model should answer questions only using the retrieved document context.

If information is not available, the system should respond appropriately instead of generating unrelated information.

6. User Interface

Build the interface using Streamlit.

Page 1: Document Upload

Features:

- Upload documents
- Display uploaded file names
- Show processing status

Page 2: Ask Questions

Features:

- Question input box
- AI-generated answer
- Source document display

Example:

Question: "What is the probation period?"

Answer: "The probation period is six months."

Source: EmployeeHandbook.pdf

Error Handling Requirements

The application must handle the following scenarios gracefully:

Invalid File Upload

If a corrupted file is uploaded:

- Show a user-friendly error message
- Prevent application crashes

No Documents Available

If a user asks a question before uploading documents:

- Display an appropriate warning message

Irrelevant Questions

If a question cannot be answered from the uploaded documents:

- Return a meaningful response indicating the information is unavailable

Large Documents

The system should successfully process large documents through chunking and retrieval mechanisms.

Project Structure

Suggested Structure:

company-knowledge-assistant/

├── app.py

├── requirements.txt

```
├── documents/
├── data/
├── utils/
|   ├── pdf_reader.py
|   ├── embedding.py
|   ├── search.py
|   └── llm.py
├── README.md
└── .env
```

GitHub Requirements

1. Create a public GitHub repository.
2. Maintain meaningful commit history.
3. Include a detailed README containing:
 4. Project overview
 5. Installation steps
 6. Features
 7. Architecture explanation
 8. Screenshots
 9. Deployment URL

Example Commit Messages:

- Initial project setup
 - Added document extraction module
 - Implemented semantic search
 - Integrated Gemini API
 - Developed Streamlit UI
 - Added error handling
 - Updated documentation
-

Deployment Requirements

Deploy the application using Streamlit Community Cloud.

Submission should include:

- GitHub Repository URL
 - Live Deployment URL
-

Bonus Feature (Optional)

Create a Document Analytics Dashboard showing:

- Total uploaded documents
 - Total processed pages
 - Query history
 - Most searched topics
 - Average response time
-

Deliverables

Submit the following:

1. GitHub Repository Link
 2. Live Application URL
 3. README Documentation
 4. Screenshots of the Application
 5. Sample Documents Used for Testing
-

Expected Outcome

The final solution should demonstrate:

- Python development skills
- Data processing using Pandas
- Document understanding
- Semantic search implementation
- GenAI integration
- Error handling
- Deployment experience

- Clean code organization